Práctica 2. Orquestación de servicios con BPMN y Flowable. Juan Carlos Montes Ronda

1. Introducción y preparación del entorno

En esta práctica hemos trabajado con dos procesos distintos dentro de una misma aplicación: la búsqueda de una oferta de vuelo y la negociación del precio de un producto. El objetivo ha sido representar su funcionamiento mediante BPMN y utilizar Flowable para ejecutar las tareas programadas en Java.

Partimos del proyecto proporcionado por el profesor. Lo abrimos como proyecto Maven en STS y comprobamos que utilizaba Java 21. La aplicación incluye Spring Boot 3.4.5, Flowable 7.2.0 y una base de datos H2 en memoria. Para arrancarla ejecutamos la clase FlightApplication y comprobamos en la consola que el servidor estaba disponible en el puerto 8080.

Antes de modificar el código enviamos una petición de reserva con el archivo authorized.json. Obtuvimos una oferta de Iberia por 300 euros, que era el comportamiento inicial esperado. A partir de ahí completamos el primer ejercicio y añadimos el segundo sin crear otro proyecto.

2. Reserva de vuelos

2.1. Implementación

El proceso inicial comprobaba si el empleado estaba autorizado y, si lo estaba, consultaba únicamente a Iberia. Para completarlo adaptamos QuoteIberia y añadimos QuoteVueling y ChooseOffer.

Los dos primeros delegados consultan a su aerolínea y guardan la disponibilidad, el precio y si ha ocurrido un fallo. Cada aerolínea utiliza variables diferentes para evitar que una consulta sobrescriba los datos de la otra. Las consultas se hacen por HTTP mediante HttpPartners, aunque los servicios simulados estén dentro de la propia aplicación.

ChooseOffer lee ambas ofertas y elige la más barata entre las disponibles con precio. Si hay empate se queda con Iberia. Si solo hay una oferta válida, la conserva, y si no hay ninguna devuelve NO_OFFER con aerolínea y precio a null. Mantuvimos los importes como BigDecimal en euros y utilizamos compareTo para compararlos.

También tratamos el fallo de una aerolínea. Si una consulta HTTP falla, su delegado guarda esa información y permite continuar. Así podemos aprovechar la oferta de la otra compañía. Además, el detalle de la respuesta diferencia un fallo de consulta de la ausencia de vuelos.

2.2. Modelo BPMN

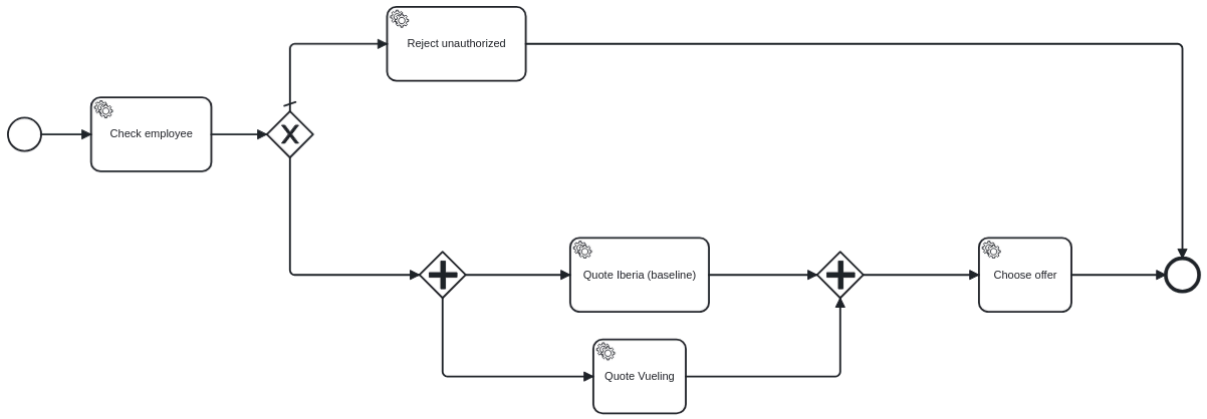


Figura 1. Diagrama del proceso de vuelos.

Abrimos el modelo inicial en bpmn.io y conservamos la comprobación del empleado y el rechazo. En el camino autorizado añadimos una pasarela paralela para abrir las dos consultas y otra para reunirlas antes de seleccionar la oferta. El rechazo sigue llegando directamente al fin.

Estas pasarelas permiten coordinar las ramas, pero no implican que las llamadas HTTP se hagan simultáneamente en nuestra configuración. Al descargar el diagrama revisamos el XML y añadimos los atributos flowable:delegateExpression para enlazar las tareas con las clases Java. Conservamos el identificador flightBooking y la información gráfica del archivo.

3. Negociación de precios

3.1. Reglas y organización del código

Para este ejercicio creamos el paquete es.ugr.dss.flight.negotiation. Separamos el catálogo y las reglas del comprador y del vendedor para poder comprobarlas antes de construir el proceso completo.

Producto	**Existencias**	**Precio inicial**	**Precio mínimo**
CAMISA	10	100 €	60 €
TRAJE	3	200 €	160 €
ZAPATOS	0	80 €	50 €

En negociación los precios se guardan como long en céntimos, tal como indica el guion. El vendedor reduce el precio un 10 % sin bajar de su mínimo. El comprador acepta si puede pagar la oferta y si esta no supera el 70 % del precio inicial. Las operaciones utilizan división entera.

Después creamos los delegados que conectan estas reglas con Flowable. ValidateNegotiation comprueba el presupuesto y el máximo de rondas; CheckStock consulta el catálogo; InitNegotiation prepara el precio inicial y la ronda 1; EvaluateBuyer evalúa la oferta; NextOffer calcula una rebaja y ApplyOffer actualiza el precio y aumenta la ronda. Otros delegados guardan los resultados finales.

3.2. Bucle y errores

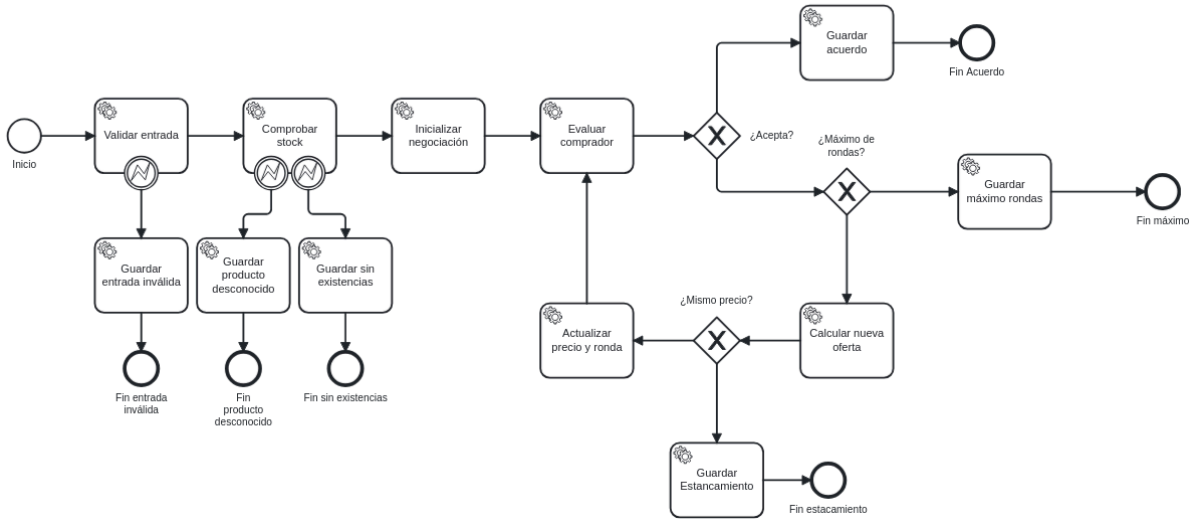


Figura 2. Diagrama de negociación, con el bucle y las salidas de error.

Dibujamos este modelo desde cero en bpmn.io. Tras validar la entrada, consultar las existencias e inicializar la negociación, se evalúa la primera oferta. A continuación se comprueba si el comprador acepta. Si rechaza, se comprueba el máximo de rondas y, si quedan rondas, se calcula una nueva oferta.

Cuando el precio nuevo es igual al anterior, el proceso termina por estancamiento. Si cambia, se aplica, se aumenta la ronda y se vuelve al comprador. No escondimos el bucle en un método Java: las decisiones y la repetición están en el diagrama.

Este orden es importante. La aceptación se comprueba antes que el límite, por lo que se puede llegar a un acuerdo en la última ronda. Una oferta repetida no cuenta como otra evaluación. Además, cada vuelta que continúa aumenta la ronda, de modo que el proceso no puede repetirse indefinidamente.

Para los problemas de entrada y catálogo utilizamos BpmnError. Añadimos un evento de error al borde de validación y dos al de existencias, con los códigos ENTRADA_INVALIDA, PRODUCTO_DESCONOCIDO y SIN_EXISTENCIAS. Sus salidas guardan ERROR, el motivo y cero evaluaciones, sin un precio negociado válido. En cambio, SIN_ACUERDO es una salida normal del proceso.

Al terminar el dibujo configuramos el XML: identificador priceNegotiation, proceso ejecutable, enlaces Java, condiciones, caminos por defecto y referencias de los errores. Mantuvimos la información gráfica para volver a abrirlo en bpmn.io.

4. Pruebas y resultados

4.1. Comprobaciones de vuelos

Primero realizamos peticiones con curl utilizando los cuatro archivos JSON del profesor. Por ejemplo:

```shell
curl -i -X POST http://localhost:8080/api/bookings \
  -H 'Content-Type: application/json' \
  --data-binary @requests/authorized.json
```

Después de completar el ejercicio obtuvimos HTTP 200 y esta respuesta:

```json
{"status":"QUOTED","employeeId":"EMP001","travelClass":"ECONOMY",
 "selectedAirline":"VUELING","totalPrice":275,"currency":"EUR",
 "detail":"Oferta seleccionada de VUELING"}
```

Frente a los 300 euros iniciales de Iberia, ahora se seleccionan los 275 euros de Vueling. Los demás resultados fueron los siguientes:

Caso	**Resultado esperado**	**Resultado obtenido**
Autorizado, EMP001 / AMS	VUELING, 275 €	Coincide
No autorizado, EMP999	REJECTED, sin aerolínea ni precio	Coincide
Destino NONE	NO_OFFER, sin aerolínea ni precio	Coincide
Destino FAIL	IBERIA, 170 €	Coincide; indica fallo de Vueling
BUSINESS, EMP002 / AMS	VUELING, 550 €	Coincide
Regreso anterior a salida	HTTP 400, sin proceso	Coincide

Los cuatro primeros casos se comprobaron por terminal y después mediante pruebas de integración. BUSINESS y las fechas también se comprobaron en estas pruebas. En FAIL, la respuesta conservó la oferta de Iberia y mostró «Fallo al consultar Vueling» en el detalle.

Ampliamos FlightIntegrationTest para exigir aerolínea y precio exactos. En rechazo y ausencia de ofertas también comprobamos los valores null. Añadimos dos pruebas unitarias de selección: empate a favor de Iberia y ausencia de ofertas. El empate utiliza 200.00 y 200 para comprobar la igualdad numérica sin cambiar las tarifas del profesor.

4.2. Comprobaciones de negociación

Antes de ejecutar el proceso probamos las reglas por separado: tres pruebas del comprador, cuatro del vendedor y cuatro del catálogo. Así comprobamos los límites de presupuesto, el descuento, el precio mínimo y la diferencia entre un producto inexistente y uno sin existencias.

Después incorporamos NegotiationIntegrationTest, proporcionado por el profesor. Inicia el proceso con RuntimeService, recupera las variables históricas y comprueba que la instancia haya terminado. Sus métodos agrupan los casos siguientes, con el descuento del 10 %:

Producto	**Presupuesto, céntimos**	**Máximo**	**Resultado comprobado**	**Precio, céntimos**	**Rondas**
CAMISA	10000	6	ACUERDO	6561	5
CAMISA	7000	6	ACUERDO	6561	5
CAMISA	7000	4	MAX_RONDAS	7290	4
CAMISA	7000	1	MAX_RONDAS	10000	1
CAMISA	6000	10	ACUERDO	6000	6
CAMISA	5900	10	PRECIO_ESTANCADO	6000	6
TRAJE	20000	10	PRECIO_ESTANCADO	16000	4
ZAPATOS	10000	6	SIN_EXISTENCIAS	Sin precio	0
LIBRO	10000	6	PRODUCTO_DESCONOCIDO	Sin precio	0

Producto	Presupuesto, céntimos	Máximo	Resultado comprobado	Precio, céntimos	Rondas
CAMISA	7000	11	ENTRADA_INVALIDA	Sin precio	0
CAMISA	7000	5	ACUERDO	6561	5
CAMISA	5900	6	MAX_RONDAS	6000	6

En la tabla, MAX_RONDAS y PRECIO_ESTANCADO son motivos de SIN_ACUERDO; los tres problemas de entrada o catálogo corresponden a ERROR. Todos los resultados coincidieron con los esperados.

Los dos últimos casos comprueban el orden de las decisiones. Se acepta una oferta en la última ronda permitida, pero un rechazo en la ronda máxima termina por MAX_RONDAS sin intentar otra rebaja.

TRAJE permite comprobar otra situación: aunque el presupuesto sea alto, el comprador solo acepta hasta 14000 y el vendedor no baja de 16000. Por eso la negociación termina sin acuerdo.

4.3. Trazas y ejecución conjunta

Además del resultado final, registramos identificadores de instancia, actividades y variables mediante HistoryService. En vuelos comprobamos que el rechazo solo ejecuta las tareas de autorización y rechazo. Para las fechas inválidas verificamos que el número de procesos no aumenta.

En negociación, la instancia 70dbcffa-d92f-11f1-a35b-caa52e600d17 mostró cinco ejecuciones del comprador y esta evolución:

```
None
Ronda:   1       2      3      4      5
Precio:  10000 → 9000 → 8100 → 7290 → 6561 céntimos
```

Esto permite explicar cómo se llega al acuerdo. Algunas actividades comparten la hora de inicio, por lo que el orden de impresión no siempre refleja exactamente el recorrido. Los elementos sin nombre aparecen como null; no son errores del proceso.

Finalmente ejecutamos toda la carpeta src/test/java desde STS:

Grupo	Número de métodos
Integración de vuelos	6
Selección de oferta	2
Comprador, vendedor y catálogo	11
Integración de negociación	5
Total	24

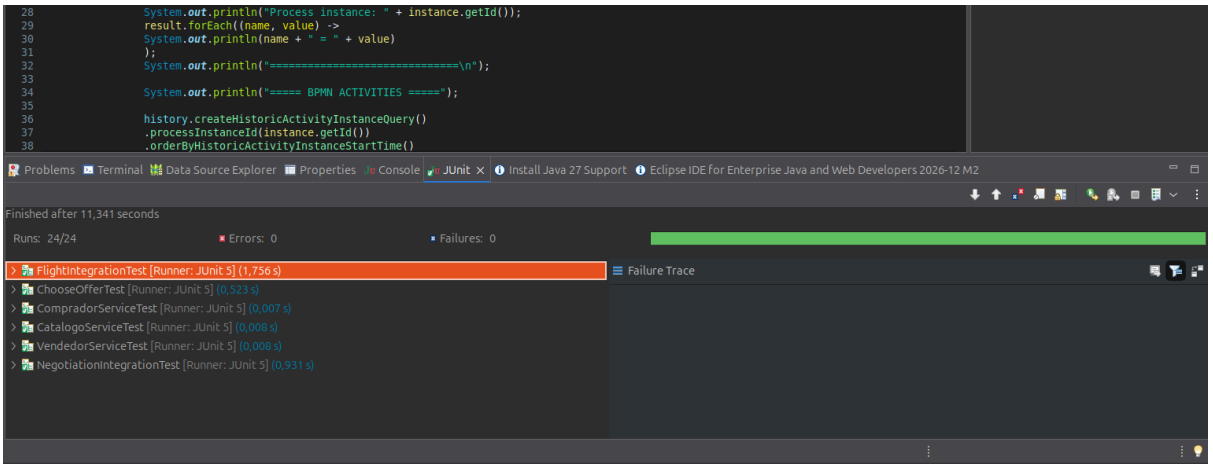


Figura 3. Las 24 pruebas terminaron sin errores ni fallos.

Esta ejecución conjunta se realizó antes de añadir los opcionales. Las ampliaciones posteriores se comprobaron por terminal, como se explica a continuación. Las respuestas completas y los identificadores de las pruebas están conservados en los documentos de apoyo.

5. Ampliaciones opcionales

5.1. API de negociación e historial de ofertas

Añadimos una entrada HTTP para iniciar el regateo. NegotiationController recibe la petición y llama a NegotiationService, que inicia el mismo proceso BPMN y recupera su resultado. La respuesta incluye el identificador de instancia para relacionarla con las trazas.

También añadimos ofertasCentimos. EvaluateBuyer guarda el precio de cada evaluación y el servicio los reúne en orden. De esta manera se devuelve el recorrido de ofertas junto al resultado final.

Lo comprobamos con esta petición:

```shell
curl -i -X POST http://localhost:8080/api/negotiations \
  -H 'Content-Type: application/json' \
  -d '{"producto":"CAMISA","presupuestoCentimos":7000,"maxRondas":6}'
```

La respuesta real fue HTTP 200:

```json
{"processInstanceId":"77248472-d932-11f1-a4cd-caa52e600d17",
 "estado":"ACUERDO","ultimoPrecioCentimos":6561,"rondasEvaluadas":5,
 "motivo":null,"ofertasCentimos":[10000,9000,8100,7290,6561]}
```

Antes de añadir la lista también probamos ZAPATOS con presupuesto 10000 y máximo seis. La API devolvió HTTP 200, estado ERROR, motivo SIN_EXISTENCIAS, precio null y cero rondas. El código HTTP indica que la petición se ha procesado; el problema de negocio aparece en el resultado.

5.2. Descuento configurable

Añadimos negotiation.descuento-porcentaje a application.properties. NextOffer lo recibe y el vendedor aplica ese porcentaje, respetando el mínimo. El valor de referencia sigue siendo 10 para mantener los resultados del guion.

Para probarlo cambiamos temporalmente la propiedad a 20, reiniciamos y enviamos la misma petición de CAMISA. Obtuvimos HTTP 200 y:

```json
{"processInstanceId":"5d64a679-d934-11f1-b204-caa52e600d17",
 "estado":"ACUERDO","ultimoPrecioCentimos":6400,"rondasEvaluadas":3,
 "motivo":null,"ofertasCentimos":[10000,8000,6400]}
```

El resultado coincide con la rebaja del 20 % y muestra el cambio en el historial. Después restauramos el descuento al 10 % y comprobamos que la propiedad quedó guardada así.

6. Conclusiones

Hemos completado dos procesos distintos utilizando Java para las tareas y BPMN para coordinar su ejecución. En vuelos trabajamos la separación y unión de consultas; en negociación, las decisiones, el bucle y los errores de negocio.

Las pruebas nos permitieron comprobar tanto los resultados como el recorrido que los produce. El historial fue especialmente útil para verificar que no se consultaban aerolíneas tras un rechazo y que las ofertas y rondas del regateo seguían las reglas del guion. Las ampliaciones permitieron además ejecutar la negociación por HTTP y consultar sus ofertas sin depender de las pruebas Java.